

Applicatie “Boekify”

Tessa Vermeulen

NOVI hogeschool

Frontend

Nova Eeken

17 mei 2026

Inhoudsopgave

Inhoudsopgave	2
Inleiding	4
Deelopdracht 1 - Functioneel ontwerp	6
De applicatie	7
Use case tabellen	8
Functionele en niet-functionele eisen	15
Inspiratiebronnen	16
Wireframes	22
Schermontwerpen	25
Deelopdracht 2 - Verantwoordingsdocument	28
Technische implementatiekeuzes	29
API:	29
De functie markAsRead:	29
Axios:	30
Button componenten	30
Het toevoegen van een nieuw boek	30
Inlog/registreer scherm	31
Het component CurrentBook	31
Het sorteren van de Library	31
Limitaties	32
Cover van het boek meesturen	32
Het toevoegen van boeken aan currentlyReadingList	32
Het geven van hartjes aan boeken	33
Het aanmaken van lijsten door de gebruiker	33
Het toevoegen van quotes uit boeken	33
Profile page/accountsettings wijzigen	33
Reading Stats en Reading Goals	33

GitHub Repository Link	34
Bijlagen	35
AI Prompts	36
Bronvermelding	50

Inleiding

In september 2025 ben ik begonnen met de Bootcamp AI Developer. De start hiervan waren de lessen Frontend. Hoewel ik al wel enige kennis van HTML en CSS schrijven had, dateert deze laatste kennis van zeker twintig jaar geleden. Het tijdperk waar de opmaak veelal met tabellen werd geregeld en ik gebruik maakte van programma's als Adobe DreamWeaver en Microsoft FrontPage, zodat ik ook direct kon zien wat het resultaat van mijn eigen, veelal simpele, geschreven codes was. JavaScript, daar had ik nog niet meegewerkt en daar zag ik eerlijk gezegd ook wel tegenop.

De afgelopen paar maanden heb ik veel geleerd, met name nieuwe CSS technieken en het schrijven van JavaScript, om zo toe te werken naar de eindopdracht die ik met dit document heb in geleverd.

In dit document wordt beschreven hoe mijn applicatie/product werkt in het functioneel ontwerp. In dit onderdeel komen onder andere de user stories, de user case tabellen, visuele inspiratie en wireframes aanbod.

Vervolgens wordt in ditzelfde document beschreven hoe ik tot bepaalde technische implementatiekeuzes ben gekomen tijdens het programmeren van mijn applicatie in het verantwoordingsdocument.

Ondanks dat ik de afgelopen maanden veel heb geleerd, merk ik wel dat dit echt de minimale basis is geweest. Tijdens het maken van mijn eindopdracht liep ik regelmatig er tegen aan dat ik in mijn functies dingen moest gaan toepassen, die niet in de lessen aanbod waren gekomen. En hoewel ik regelmatig mijn vragen heb gesteld, lukte het de SME-ers ook niet altijd om te helpen.

Vanuit NOVI werd er aangeraden om al tijdens de lessen te werken aan je eindopdracht. En hoewel ik begrijp waarom, was dit voor mij een lastige opgave. Juist omdat het schrijven van JavaScript zoveel tijd van mij vraagt, ging mijn aandacht vooral uit naar de huiswerkopdrachten om hier mee te oefenen. En met de huiswerkopdrachten was ik vaak toch de hele week bezig op de momenten dat ik de energie er voor had. Hierdoor bleef er weinig tijd voor mij over om ook aan de eindopdracht te werken, juist ook omdat de ideefase pas halverwege oktober 2025 ingeleverd moest worden, waardoor er in de lessen zelf pas tegen die tijd informatie over kwam. Persoonlijk had ik dit liever eerder ingeleverd.

Ik heb uiteindelijk gekozen om voor mijn gehele applicatie de API van NOVI te gebruiken, waardoor ik deze zelf helemaal moest inrichten. Nu staan er voorbeelden van `json` files bij de documentatie. Die heb ik ook zeker gebruikt, echter liep ik er ook tegen aan dat niet alle informatie duidelijk beschreven was. Voor een student die de Bootcamp Fullstack voltijd doet, zal dit duidelijk zijn, maar voor mij (zonder enige backend ervaring) was het goed werkend krijgen van de API een zeer grote klus. Ik heb meermalen mijn `json` file moeten aanpassen, en daardoor ook mijn code weer moeten aanpassen en werkend zien te krijgen. Daarnaast heb ik geprobeerd de permissions aan te passen en `anonymous` weg te halen. Echter werkte toen mijn applicatie niet meer. Ik ben me bewust van de gevaren hiervan mocht de website ooit echt online komen, maar dit is een backend ding, waar ik geen weet van heb. Ik had het fijn gevonden wanneer er de optie was geweest om hier een of twee lessen in te krijgen, net als toen met GitHub.

Een ander punt waar ik tegen aanliep was het maken van dit document. Ik weet van mij zelf dat ik het beste leer met duidelijke voorbeelden, en voor het maken van dit document ontbrak het daar voor mij aan. Ik had geen idee hoe te beginnen, de "opdrachten" waren voor mij te summier beschreven en voor mij ontbrak het aan duidelijke verwachtingen. Ik heb natuurlijk vragen gesteld,

maar er werd ook aangegeven dat wat betreft de eindopdracht niet alle vragen beantwoord konden worden.

Waar ik ook tegen aanliep is de bronvermelding. Ik wist niet goed hoe ik dat met de APA-regels kon toepassen. Het gaat in dit document vooral om screenshots die ik heb gemaakt van websites en iPad apps en hoewel Scribbr (www.scribbr.nl) heel duidelijk liet zien hoe je citaten en boeken op een juiste manier in je bronvermelding kon noteren, was dit met een screenshot voor een website geheel niet te vinden.

Al met al heb ik keihard gewerkt om tot dit eindproduct te komen, zowel in de applicatie, als met dit document.

Deelopdracht 1 - Functioneel ontwerp

De applicatie

Ben jij ook zo dol op lezen? Zie je soms een boek in de boekwinkel of bibliotheek waarvan je niet zeker meer weet of je deze in je bezit hebt, of al hebt gelezen? Dit overkomt mij regelmatig. Dan sta ik in een boekwinkel en zie ik een boek waarvan ik denk *leuk!* die neem ik mee. Vervolgens kom ik thuis, heb ik het boek al in m'n boekenkast staan. Of ik ben bij de bibliotheek en zie een boek wat me interessant lijkt, leen het en vervolgens begin met lezen en merk dat het verhaal me bekend voorkomt. Daarom ga ik een applicatie programmeren waarbij ik mijn eigen thuisbibliotheek kan gaan bijhouden. Op deze manier heb ik op een gebruiksvriendelijke manier een overzicht van mijn boeken en de boeken die ik gelezen heb. In deze applicatie zijn de belangrijkste kernfunctionaliteiten:

Als admin/lid kan ik me inloggen/registreren, zodat gegevens niet door derden zomaar kunnen worden veranderd.

Als admin kan ik mijn eigen boeken toevoegen, zodat de thuisbibliotheek in de app komt te staan.

Als admin/lid kan ik bijhouden welke boeken ik heb gelezen, zodat de kans op dubbele boeken kopen/lezen geminimaliseerd wordt.

Als admin/lid kan ik reviews over boeken schrijven, zodat ik weet hoe ik het boek vond, en deze eventueel kan aanbevelen aan anderen om te lezen.

Use case tabellen

Use Case 1	Inloggen
Actor	De gebruiker
Trigger	De gebruiker klikt op een knop inloggen die in het scherm staat.
Preconditie	De gebruiker heeft de app geopend.
Main Success Scenario	1. De gebruiker vult zijn username in. 2. De gebruiker vult zijn wachtwoord in. 3. De gebruiker klikt op de knop <i>inloggen</i>.
Postconditie	De gebruiker ziet zijn eigen startpagina.

Alternatief Scenario 1A	Verkeerde username
Stappen	1. Het systeem toont de melding dat de username onjuist is. 2. De gebruiker voert de juiste username in.
Postconditie	De gebruiker gaat verder met stap 2 (vult zijn wachtwoord in).

Alternatief Scenario 2A	Verkeerd wachtwoord
Stappen	1.Het systeem toont de melding dat het wachtwoord onjuist is. 2.De gebruiker voert het juiste wachtwoord in of klikt op de link wachtwoord vergeten.
Postconditie	De gebruiker gaat verder met stap 2 (drukken op de knop inloggen).

Use Case 2	Boeken toevoegen
Actor	De gebruiker
Trigger	De gebruiker klikt op een knop boek toevoegen in het scherm.
Preconditie	De gebruiker is ingelogd.
Main Success Scenario	1.De gebruiker vult de titel in. 2.De gebruiker vult de auteur in. 3.De gebruiker voegt een afbeelding van de cover toe. 4.De gebruiker vult het genre in. 5.De gebruiker vult het ISBN nummer in. 6.De gebruiker klikt op de knop ✓.
Postconditie	Het boek is toegevoegd.

Alternatief Scenario 1A	De gebruiker heeft niet toegestane karakters gebruikt
Stappen	1.Het systeem geeft een melding dat er niet toegestane karakters zijn gebruikt in het invoerveld en maakt de knop ✓ disabled. 2.De gebruiker verwijdert de niet toegestane karakters. 3.Het systeem laat de melding verdwijnen maakt de knop ✓ weer enabled.
Postconditie	De gebruiker kan verder met stap 2 (auteur invoeren).

Alternatief Scenario 3A	Afbeeldingsbestand is te groot
Stappen	1.Het systeem geeft een melding dat de bestandsgrootte van de afbeelding te groot is. 2.Het systeem geeft de maximale bestandsgrootte aan. 3.De gebruiker verkleint de bestandsgrootte. 4.De gebruiker upload de nieuwe afbeelding.
Postconditie	De gebruiker kan verder met stap 4 (genre invullen).

Alternatief Scenario 3B	Afbeeldingsformaat wordt niet herkend
Stappen	1. Het systeem geeft een melding dat het afbeeldingsformaat niet wordt herkend. 2. Het systeem geeft aan welke formaten worden ondersteund (jpg, jpeg, gif, png, svg). 3. De gebruiker past het formaat van de afbeelding aan, aan een ondersteunend formaat. 4. De gebruiker upload de nieuwe afbeelding.
Postconditie	De gebruiker kan verder met stap 4 (genre invullen).

Use Case 3	Bijhouden welk boek gelezen is
Actor	De gebruiker
Trigger	De gebruiker klikt op een knop + in het scherm.
Preconditie	De gebruiker is ingelogd.
Main Success Scenario	1. De gebruiker krijgt een zoekveld te zien. 2. De gebruiker typt de titel van het boek in. 3. Het boek verschijnt in een lijst. 4. De gebruiker klikt op het boek in de lijst. 5. De pagina van het boek wordt geopend. 6. De gebruiker klikt op een knop gelezen.
Postconditie	Het boek is toegevoegd aan de lijst gelezen boeken.

Alternatief Scenario 2A1	De titel van het boek verschijnt niet in de lijst
Stappen	1.Het systeem laat het boek niet zien wat de gebruiker heeft ingetypt. 2.De gebruiker checkt de spelling van het de titel. 3.De gebruiker past de titel aan.
Postconditie	De gebruiker kan verder met stap 3

Alternatief Scenario 2A2	De titel van het boek verschijnt niet in de lijst
Stappen	1.Het systeem laat het boek niet zien wat de gebruiker heeft ingetypt. 2.De gebruiker checkt de spelling van het de titel. 3.De gebruiker past de titel aan. 4.De titel verschijnt nog steeds niet in de lijst.
Postconditie	De gebruiker moet het boek toevoegen.

Use Case 4	Een review schrijven
Actor	De gebruiker
Trigger	De gebruiker heeft het boek toegevoegd aan de gelezen lijst.
Preconditie	De gebruiker is ingelogd.
Main Success Scenario	1. De gebruiker kiest het boek waar hij een review aan wil geven. 2. De gebruiker klikt op het boek. 3. De gebruiker klikt op een knop <i>review toevoegen</i>. 4. De gebruiker klikt op het aantal sterren (max. 5). 5. De gebruiker schrijft in het tekstvak zijn review. 6. De gebruiker klikt op een knop <i>submit</i>.
Postconditie	Het review is aan het boek toegevoegd.

Alternatief Scenario 5A	Het aantal toegestane karakters is overschreden
Stappen	1. Het systeem geeft aan dat het aantal toegestane karakters is overschreden en <i>unabled</i> de submit button. 2. De gebruiker past het aantal karakters in zijn review aan. 3. Het systeem <i>enabled</i> de submit button weer.
Postconditie	De gebruiker kan verder met stap 6.

Alternatief Scenario 5B	Er worden niet toegestane karakters gebruikt
Stappen	1. Het systeem geeft aan dat er niet toegestane karakters worden gebruikt en unabled de submit button. 2. De gebruiker verwijdert de niet toegestane karakters in zijn review. 3. Het systeem enabled de submit button weer.
Postconditie	De gebruiker kan verder met stap G.

Functionele en niet-functionele eisen

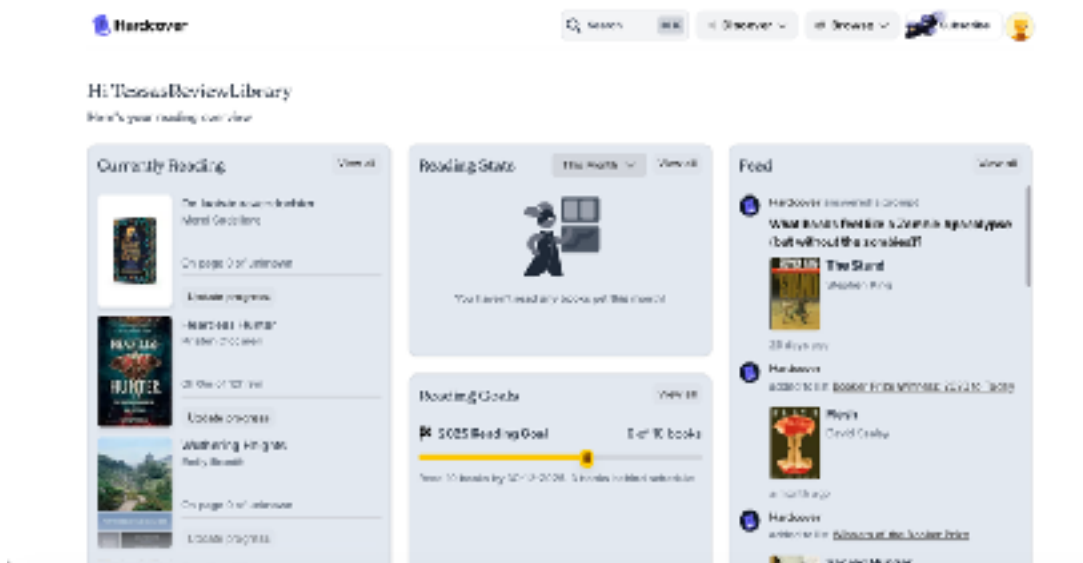
(Wat moet mijn applicatie kunnen?)

Functionele eisen / niet-functionele eisen

1. Als gebruiker moet ik kunnen inloggen.
2. Als gebruiker moet ik handmatig boeken kunnen toevoegen met de titel, auteur, cover, uitgever, jaar, aantal pagina's, beschrijving, genres, taal, format, isbn, publicatiejaar.
3. Als gebruiker moet ik toegang krijgen tot een database met boeken om boeken te kunnen toevoegen aan mijn thuisbibliotheek.
4. Als gebruiker moet ik reviews bij de boeken kunnen schrijven.
5. Als gebruiker moet ik kunnen bijhouden welke boeken ik heb gelezen.
6. Als gebruiker moet ik kunnen aangeven hoeveel sterren ik een boek geef.
7. Als gebruiker wil ik kunnen aangeven welk boek ik momenteel aan het lezen ben.
8. Als gebruiker wil ik lijsten kunnen maken zoals bijv. een TBR, favorieten, DNF.
9. Als gebruiker wil ik kunnen bijhouden hoeveel ik gelezen heb.
10. Als gebruiker wil ik bij gelezen boeken quotes kunnen noteren.
11. Als gebruiker wil ik de tekst in de app makkelijk kunnen lezen.
12. Als gebruiker wil ik de mogelijkheid hebben om een handleiding te kunnen lezen zodat ik de app beter begrijp.
13. Als gebruiker wil ik dat wanneer ik een boek zoek, de lijst met boeken snel wordt geladen.
14. Als gebruiker wil ik dat de tekst op de achtergrond goed te lezen is.
15. Als gebruiker wil ik dat de app qua lay-out rustig toont.
16. Als gebruiker wil ik dat de verschillende pagina's vlot openen.
17. Als gebruiker wil ik door op 1 knop te klikken een boek kunnen toevoegen aan mijn leeslijsten.
18. Als gebruiker wil ik dat de webapplicatie goed beschikbaar is, ook als er veel bezoekers zijn.
19. Als gebruiker wil ik boeken uit mijn thuisbibliotheek kunnen verwijderen dat ze in mijn bezit zijn (ze blijven dan nog wel in de gelezen boekenlijst).
20. Als gebruiker wil ik dat de lay-out netjes blijft, ook al is mijn scherm kleiner.
21. Als gebruiker wil ik dat mijn privacy gewaarborgd blijft, dus e-mail adres etc niet zichtbaar kunnen maken.
22. Als gebruiker wil ik kunnen schakelen tussen dag-/nachtmodus
23. Als gebruiker wil ik mijn gelezen boeken kunnen importeren d.m.v. bijv. een download van Goodreads.
24. Als gebruiker wil ik mijn data kunnen exporteren.
25. Als gebruiker wil ik mijn account settings kunnen wijzigen.

Inspiratiebronnen

Figuur 1



Noot. Screenshot Hardcover (<https://hardcover.app/>)

De manier waarop de beginpagina van deze webapplicatie Hardcover (<https://hardcover.app/>) in de afbeelding hierboven na inloggen is vormgegeven vind ik rust uitstralen en overzichtelijk. Dit komt door het gebruik van de blokken, de ruimte er tussen en weinig accent kleuren. Het gebruik van de indeling van de begin pagina na inloggen, wil ik hierom gebruiken en ook minimale accentkleuren.

Figuur 2

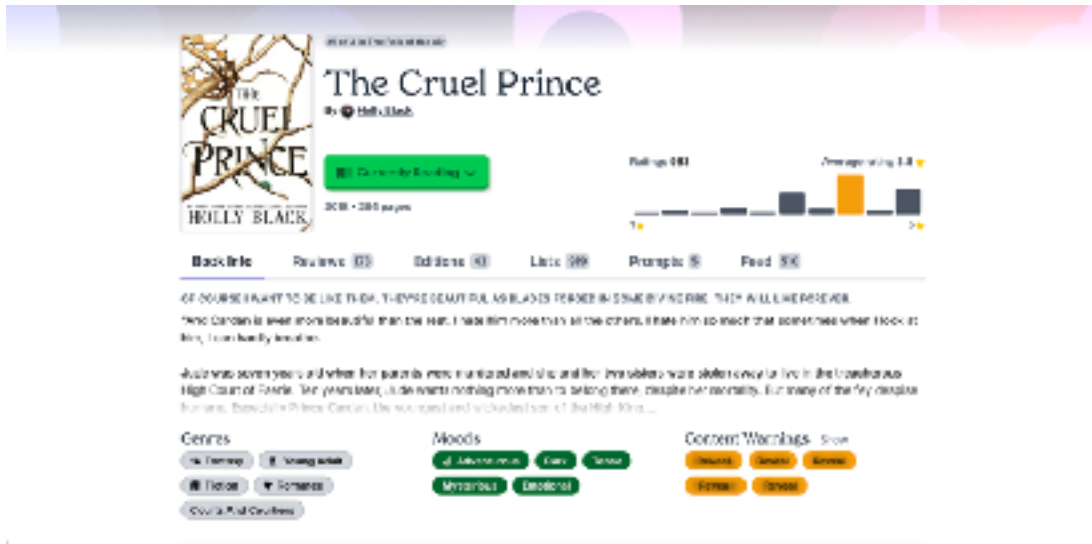


Noot. Screenshot Goodreads (<https://www.goodreads.com/>)

Eigenlijk is deze lay-out van de website Goodreads (<https://www.goodreads.com/>), compleet het tegenovergestelde van de vorige inspiratiebron. Dit is voor mij een reminder hoe ik het *niet* wil.

Wat wel een inspiratiebron is, is de navigatie; geen grote knoppen als navigatie, maar een link met een hover-kleur en een icoontje voor je profiel. Deze manier van een navigatie is voor mij een optie.

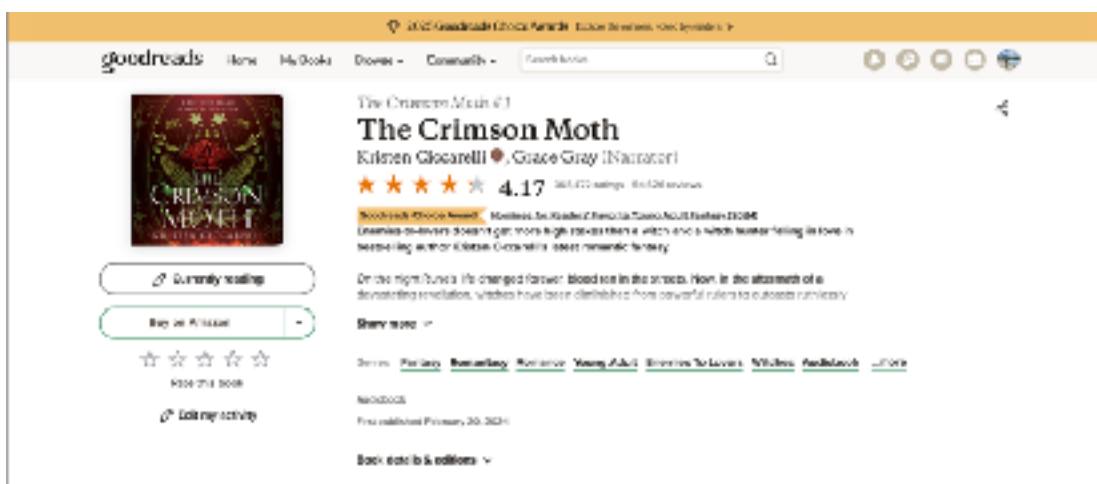
Figuur 3



Noot. Screenshot Hardcover (<https://hardcover.app>)

De afbeelding hierover van de pagina met informatie over een boek (Hardcover website, <https://hardcover.app/>) vind ik qua lay-out overzichtelijk. De onderdelen die ik (ooit) wil gaan toepassen zijn: de coverafbeelding, de titel, de auteur, het jaar van uitgave, het aantal pagina's, informatie over het boek, de genres, de moods en de content warnings.

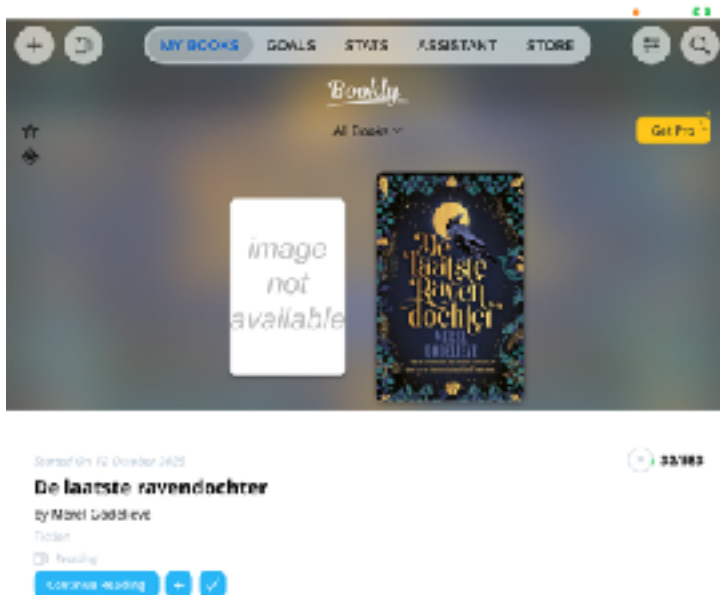
Figuur 4



Noot. Screenshot Goodreads (<https://www.goodreads.com>)

Wat ik bij deze boekpagina van Goodreads (<https://www.goodreads.com/>) dan weer goed vind en wil proberen toe te passen is de "Show more" en de Bookdetails & editions uitklapper.

Figuur 5



Noot. Screenshot Bookly iPad App

Deze boekenlijst (Bookly iPad App) is hoe ik het niet wil. De covers van de boeken draaien als een carrousel in het rond. Wel is direct duidelijk hoe je een boek snel kan toevoegen door het plus-teken, dus die wil ik gaan proberen toe te passen. Op zich is de manier waarop duidelijk is op welke pagina je bent ook een optie om toe te passen.

Figuur 6



De afbeelding links (Hardcover iPad App) laat zien wat ik zoveel mogelijk wil voorkomen: dat wanneer het scherm kleiner wordt, de tekst buiten het kader valt, zoals hier te zien is.

Ook hier is weer wel overzichtelijk te zien wat elk vak inhoudt. Dit is iets wat ik wil proberen door mijn hele applicatie toe te passen.

Noot. Screenshot Hardcover iPad App

Figuur 7



Noot. Screenshot Book Tracker iPad App

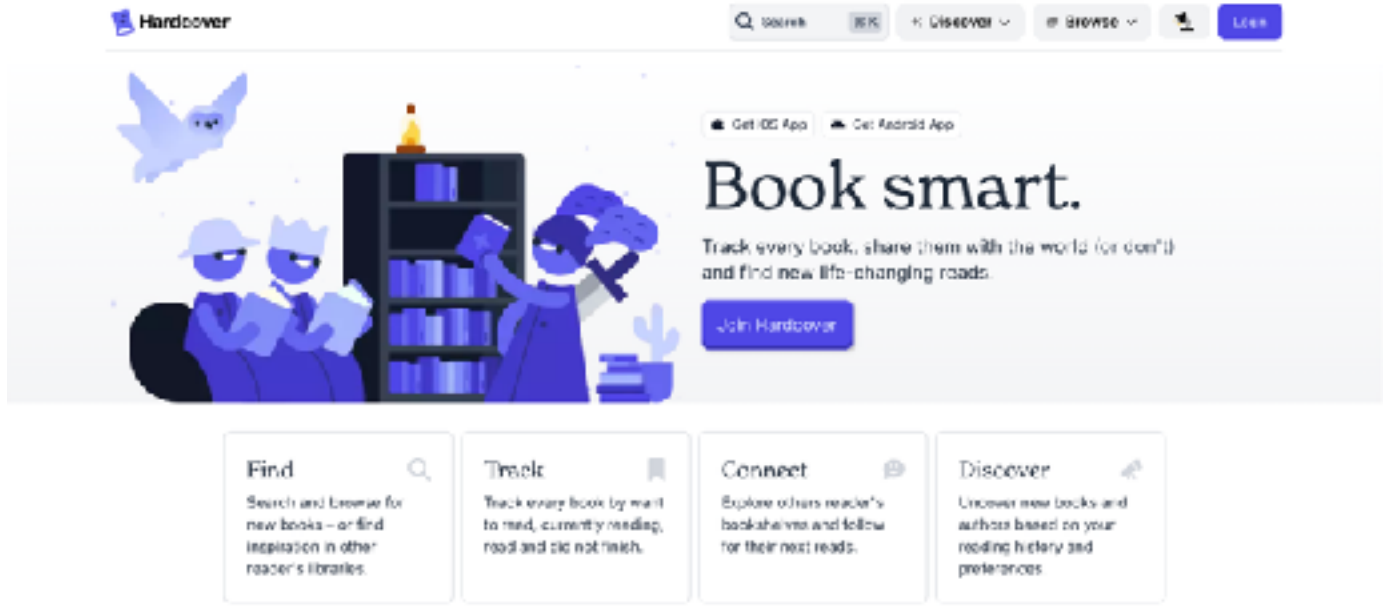
Figuur 8



Noot. Screenshot Book Tracker iPad App

In de afbeelding links (Book Tracker iPad App) vind ik het toevoegen van een quote op deze manier overzichtelijk. Ook hier is er weer duidelijk onderscheid tussen alle secties, je kan de quote een kleur geven, de pagina toevoegen en de cover van het desbetreffende boek. Ik weet nog niet of ik dit als pop-up wil laten gebeuren, of gewoon in de pagina het wil laten doen.

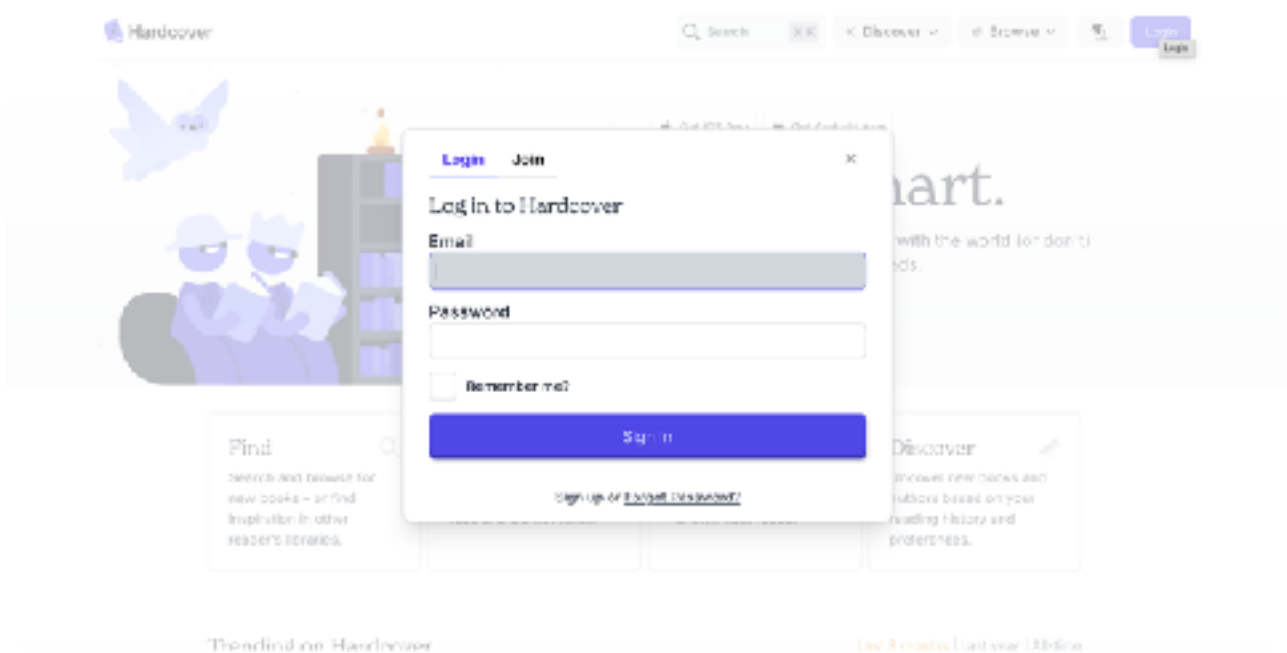
Figuur 9



Noot. Screenshot Hardcover (<https://hardcover.app>)

Deze startpagina van Hardcover (<https://hardcover.app/>) ziet er overzichtelijk en rustig uit. Deze webapplicatie is niet zozeer een thuisbibliotheekstelsel, maar meer een communitystelsel. Ik vind de basis mooi voor het thuisbibliotheekstelsel. Het gebruik van blokken maakt het overzichtelijk, dat is iets waar ik voor wil zorgen. Of het helemaal op deze manier gaat, weet ik nog niet zeker.

Figuur 10

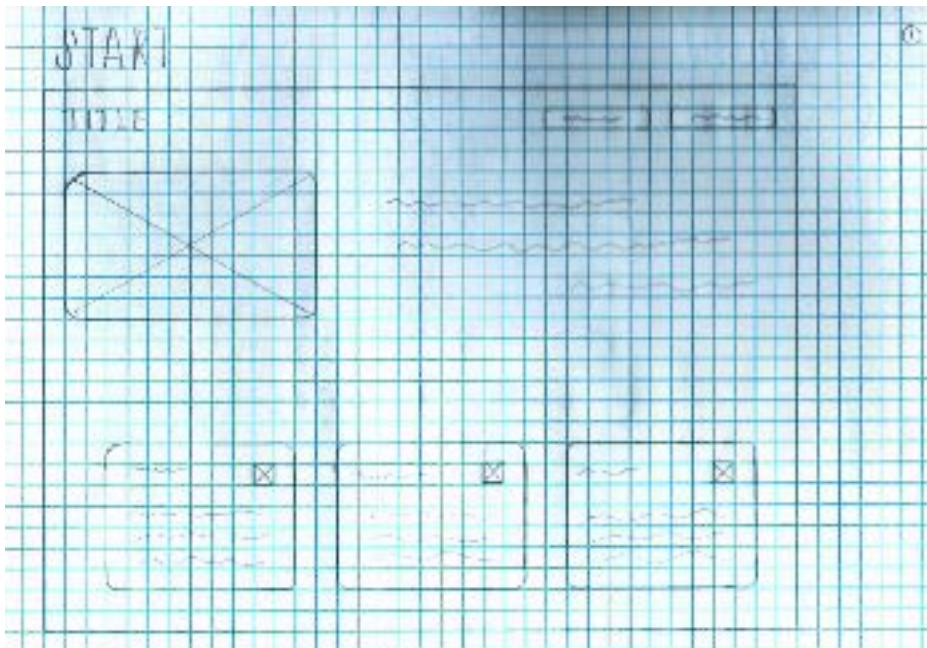


Noot. Screenshot Hardcover (<https://hardcover.app>)

Het inlogvenster van Hardcover (<https://hardcover.app/>) is ook weer duidelijk en overzichtelijk. Ik weet nog niet zeker of ik een popup pagina er van maak, of dat ik de pagina gewoon laat openen in de browser.

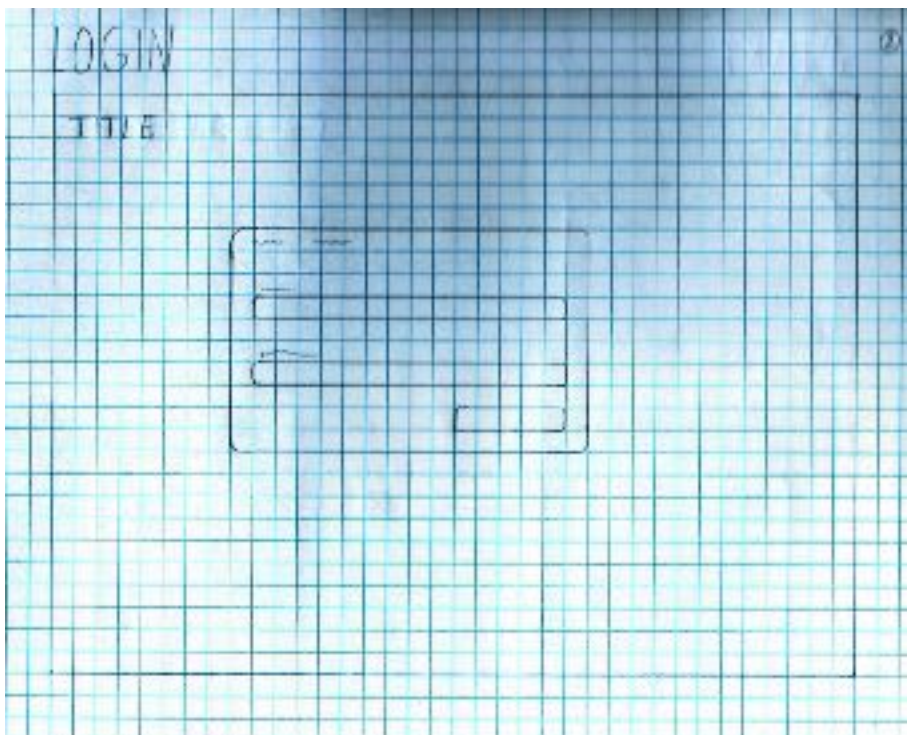
Wireframes

Figuur 11



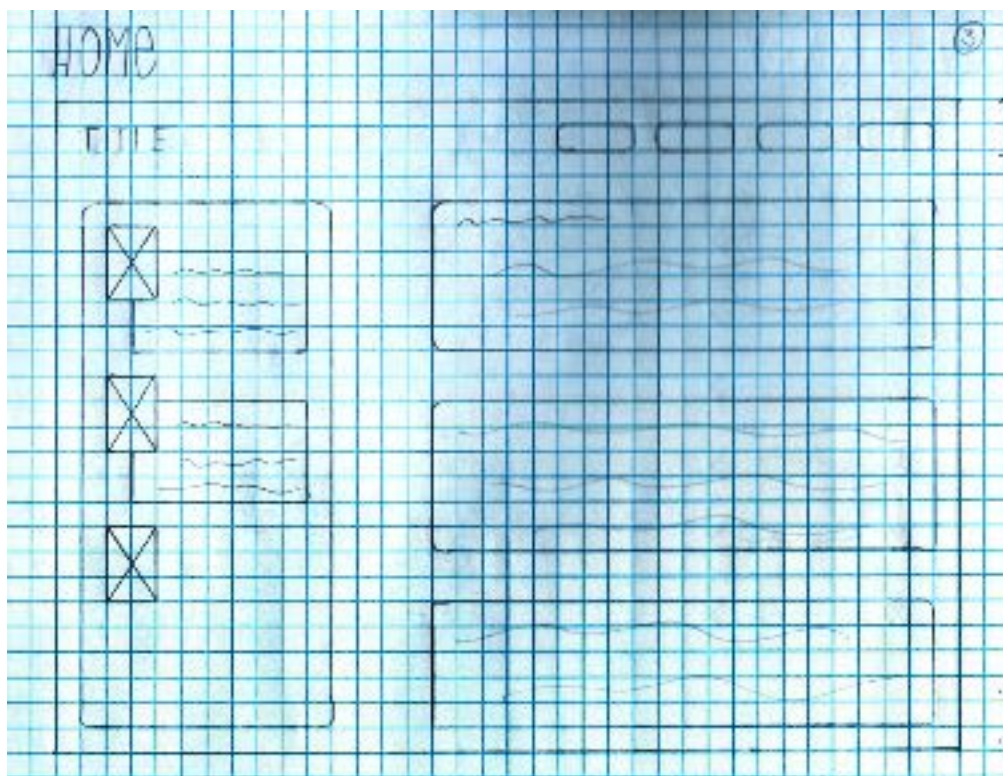
Noot. Scan wireframe startpagina

Figuur 12



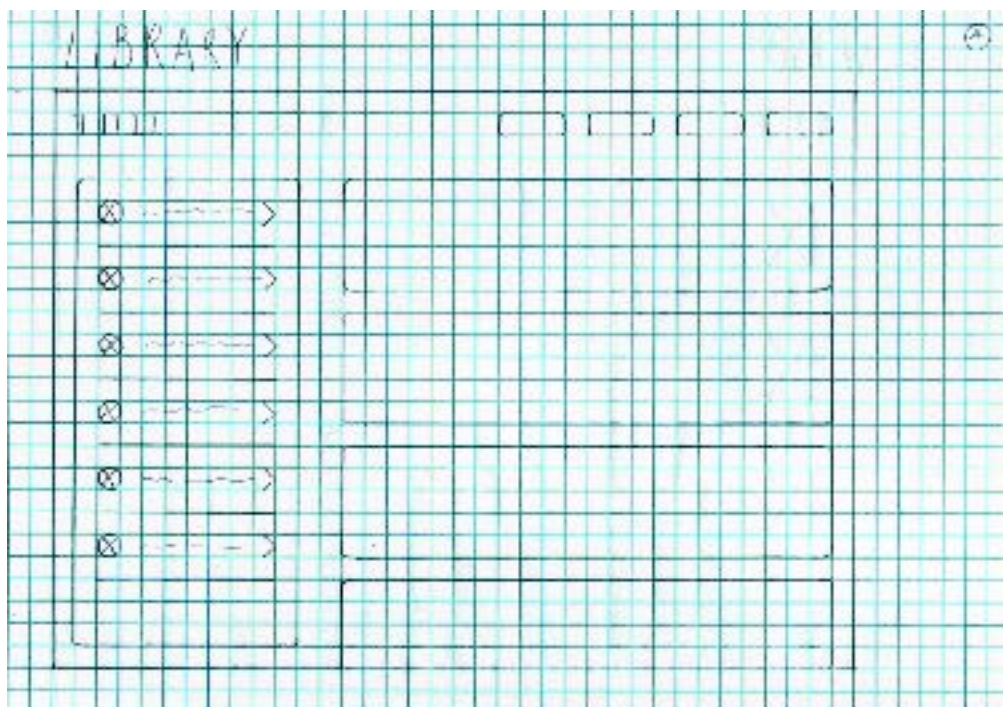
Noot. Scan wireframe loginpagina

Figuur 13



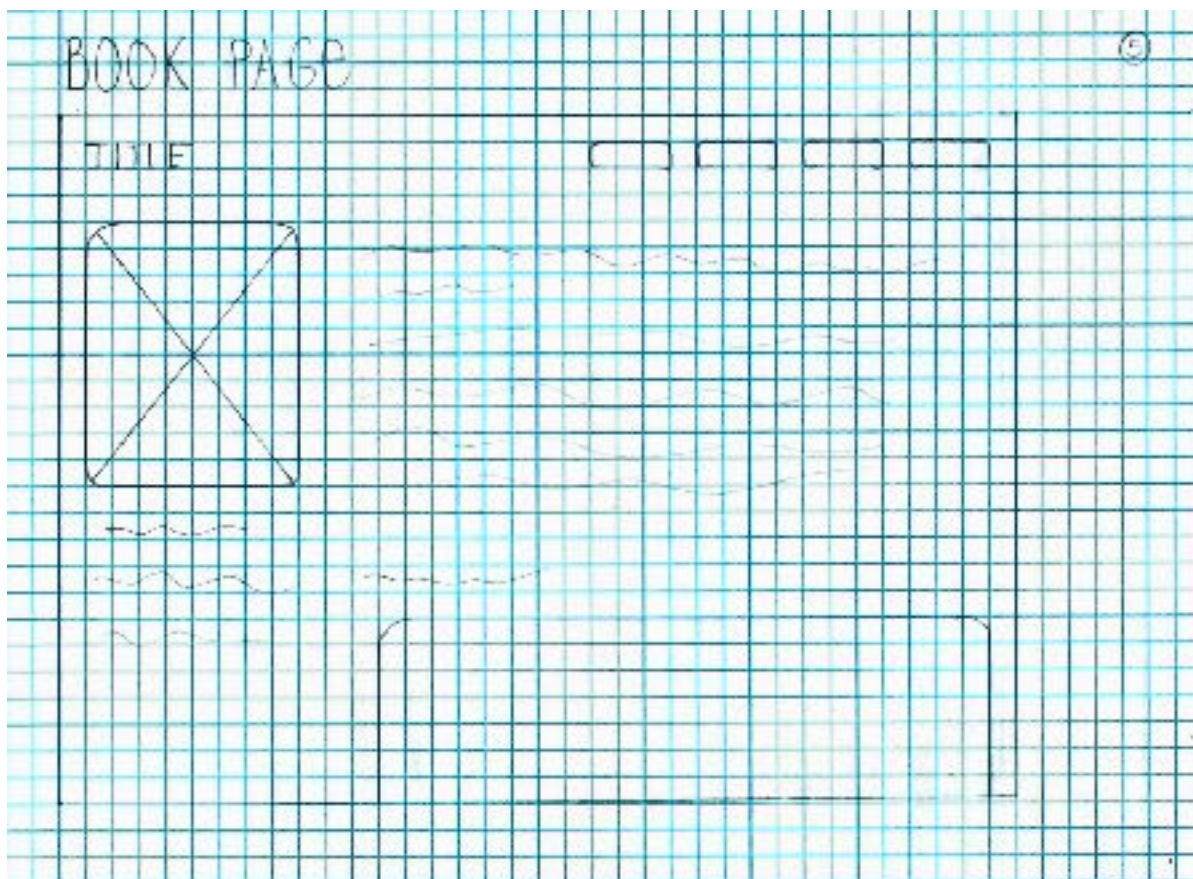
Noot. Scan wireframe homepagina

Figuur 14



Noot. Scan wireframe library pagina

Figuur 15



Noot. Scan wireframe book pagina

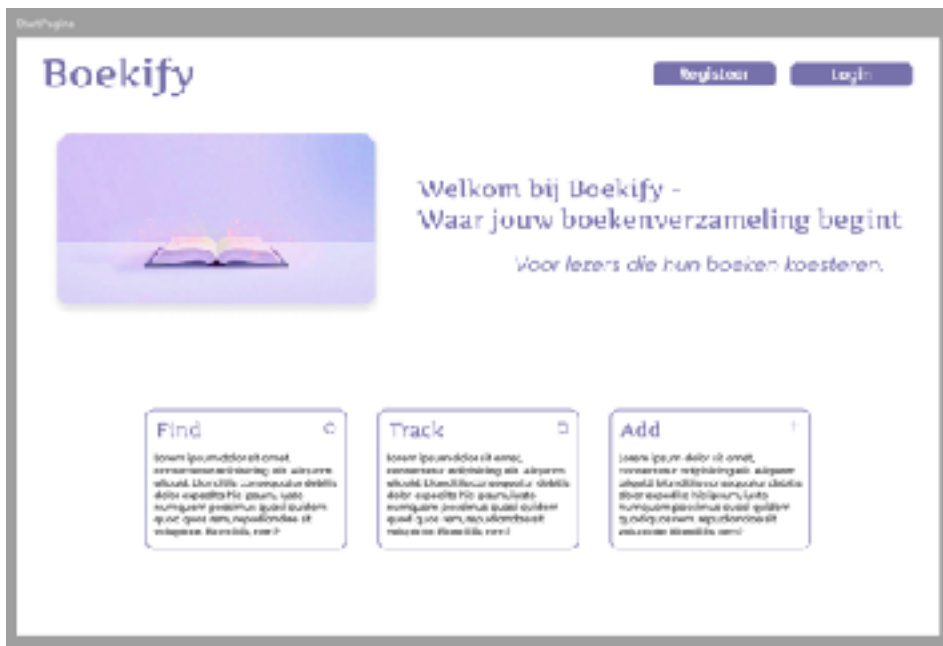
Schermontwerpen

Het originele project in Figma:

<https://www.figma.com/files/team/1567150824432341191/project/520885404/Eindopdracht---Thuisbieb?fuid=1567150820781682028>

Wachtwoord: eindopdracht-novi

Figuur 16



Noot. Screenshot Figma start pagina

Figuur 17



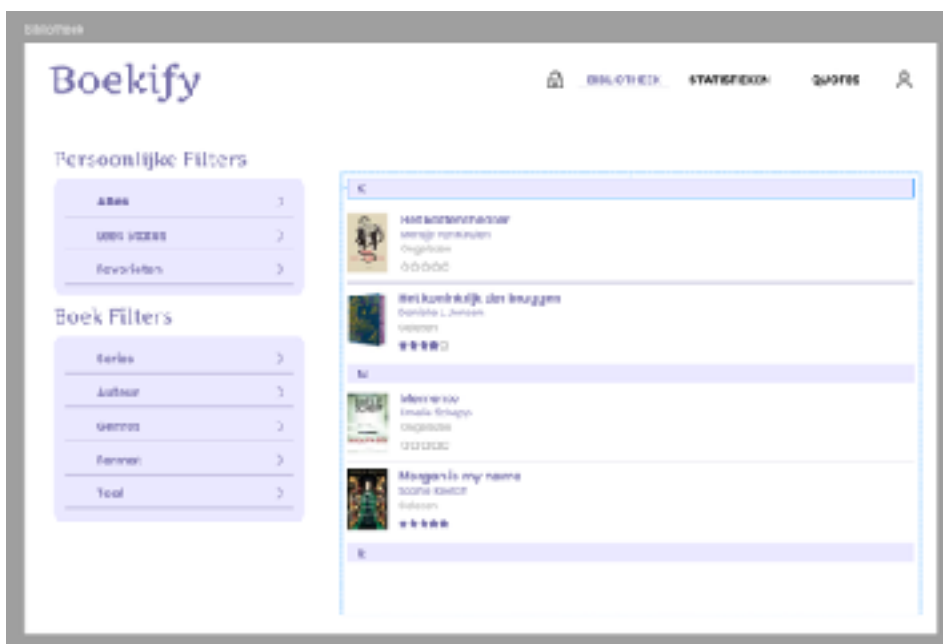
Noot. Screenshot Figma Inlogpagina

Figuur 18



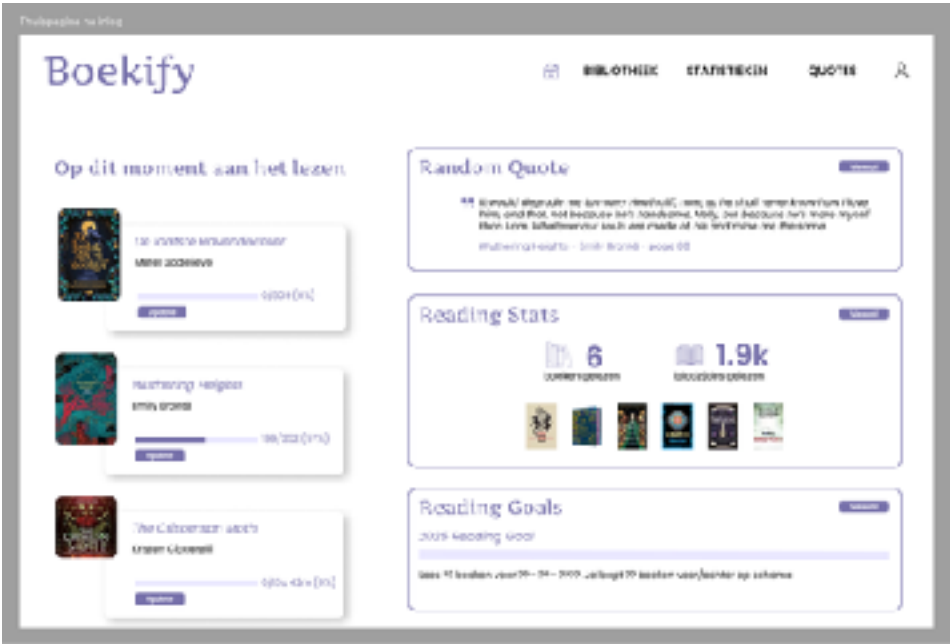
Noot. Screenshot Figma Registratiepagina

Figuur 19



Noot. Screenshot Figma Bibliotheekpagina

Figuur 20



Noot. Screenshot Figma Homepagina

Deelopdracht 2 - Verantwoordingsdocument

Technische implementatiekeuzes

API:

In eerste instantie wilde ik voor het inloggen de API van NOVI gebruiken en voor de boekenlijst/library de API van **openbook.org**. De documentatie van de **openbook.org** API was voor mij moeilijk te doorgronden. Ik kreeg hierdoor de API ook niet werkend. Daarom heb ik ervoor gekozen om de NOVI API in zijn geheel te gebruiken en dan ook mijn eigen json-file te maken. Hierdoor weet ik nu dat je eerst begint met het aanmaken van "collections", waarbij je ook de "permissions" moet toevoegen ter beveiliging. In de verschillende lijsten kan je terug refereren naar andere lijsten, die nodig zijn om een lijst goed op te halen. Vervolgens maak je de "data" aan. En deze luistert nauw met de informatie die je in de "collections" hebt genoteerd. En niet alleen luistert dit nauw in de json-file zelf, maar ook in de functies waar je de API aanspreekt. Zo liep ik er tegen aan dat ik wanneer ik op de admin account was ingelogd wel kon aangeven dat ik een boek gelezen had en van de ene lijst naar de ander kon plaatsen, maar een nieuw geregistreerde gebruiker kon dit niet. Als eerste moest ik ervoor zorgen dat wanneer een nieuwe gebruiker registreert, de lijsten `readList` en `currentlyReadingList` aangemaakt moesten worden. Vervolgens bleek dat ik nog een `readListItem` array en een `currentlyReadingItem`

array moest aanmaken, waar juist daar de boeken in opgeslagen moesten gaan worden voor de desbetreffende lijsten (relationele normalisatie). Hoewel ik de documentatie (<https://novi-backend-api-wgsgz.ondigitalocean.app/documentation/1-Overview>) voor het maken van een json file, meer malen heb doorgenomen en had gekeken naar het voorbeeld van de bibliotheek, heb ik de informatie over relationele normalisatie niet gezien. Het was fijn dat een van de SME-ers mij deze documentatie toe had gestuurd.

Wat ik wel prettig vind, is dat je het ophalen van data geheel in eigen beheer houdt. Je bent niet afhankelijk van een derde partij, die zomaar ineens kan besluiten om de code te gaan veranderen. Daarnaast heb ik geprobeerd de permissions aan te passen en `anonymous` weg te halen. Echter werkte toen mijn applicatie niet meer. Ik ben me bewust van de gevaren hiervan mocht de website ooit echt online komen, maar dit is een backend ding, waar ik geen weet van heb.

DE FUNCTIE MARKASREAD:

Voordat ik deze functie had geschreven, had ik eerst al verschillende manieren geprobeerd. Van elk API verzoek een aparte functie, tot een behoorlijk lange functie voor een selectbox waarin alle verzoeken verwerkt werden. In eerste instantie wilde ik ook graag de selectbox gebruiken, waarin de gebruiker zowel het boek als gelezen, als op dit moment aan het lezen kon markeren. Voor deze manier heb ik AI gevraagd mijn leraar te zijn en had ik met de tips een functie geschreven. Echter bleek later dat ik dingen in mijn `data.json` file moest gaan aanpassen (relationele normalisatie).

Figuur 21



```
"collections": [
  {
    "name": "readList",
    "fields": [
      {
        "name": "id",
        "type": "number",
        "required": true
      },
      {
        "name": "bookId",
        "type": "reference:books",
        "required": true
      },
      {
        "name": "authorId",
        "type": "reference:authors",
        "required": true
      },
      {
        "name": "userId",
        "type": "reference:users",
        "required": true
      },
      {
        "name": "ratingId",
        "type": "reference:ratings",
        "required": false
      }
    ]
  },
  {
    "name": "currentlyReadingList",
    "fields": [
      {
        "name": "id",
        "type": "number",
        "required": true
      },
      {
        "name": "bookId",
        "type": "reference:books",
        "required": true
      },
      {
        "name": "userId",
        "type": "reference:users",
        "required": true
      }
    ]
  }
],
"permissions": {
  "GET": [
    "anonymous",
    "admin"
  ]
}
```

Noot. Screenshot code `data.json`

Hierdoor werd deze functie voor mij onbruikbaar en kreeg ik hem niet meer werkend. Ik heb toen de functie compleet herschreven tot de huidige functie. Ik ben eerst aan de gang gegaan met het markeren van het boek als gelezen. Dit heeft me behoorlijk veel tijd gekost, ik liep telkens vast op een melding dat het boek niet uit de lijst verwijderd kon worden. Uiteindelijk bleek ik `'Authorization': `Bearer ${localStorage.getItem('JWT')}`` uit de headers te moeten verwijderen, waardoor het nu wel lukt. Dit was een uitprobeersel die goed uitpakte, maar ik heb uiteindelijk wel aan Google gevraagd waarom ik dit uit de header moest halen om hier meer over te weten te komen. Het AI overzicht van Google gaf drie redenen die konden meespelen:

1. Het verzoek had eigenlijk een andere HTTP-methode moeten zijn.
2. Conflicterende of dubbele headers door een HTTP client.
3. Oplossen van CORS-problemen (Cross-Origin Resource Sharing).

Alle drie punten die minimaal of niet behandeld zijn in de les.

AXIOS:

Voor het ophalen van de API data heb ik gebruik gemaakt van de NPM package `Axios`. Deze manier hebben we veelal gebruikt tijdens de lessen, maar ik heb het verschil opgezocht met `fetch` (de andere methode die benoemt is). `Axios` heeft veel meer functies, die je bij `fetch` zelf zou moeten schrijven. Wel heb ik geëxperimenteerd om `axios` op verschillende manieren in een functie te gebruiken. Zo begin ik soms met `"async function..."` en soms met `"const handle... = async (e) => {..."`. Dit heeft dan ook met name te maken of ik gewoon iets van de API wil ophalen, of wanneer er een `onSubmit` of `handleSubmit` nodig was. Ik heb gemerkt dat ik `axios` voor het ophalen van API data een fijne manier van code schrijven vind. Ik ervaar het als overzichtelijker en vind het heel fijn dat in `axios` functies zitten, die ik anders zelf had moeten schrijven.

BUTTON COMPONENTEN

De meest logische keuze zou zijn geweest om één button component te maken, die je gebruikt voor al je buttons in de applicatie. Hier ben ik van afgeweken. Ik heb twee button componenten gemaakt. En wel om de heel simpele reden dat ik het niet voor elkaar kreeg om de grootte van de buttons aan te passen wanneer ik één component gebruikte. Ik wilde hier op dit moment niet te lang bij stil staan, maar dit is zeker iets waar ik me nog in wil gaan verdiepen en het wil gaan verbeteren.

HET TOEVOEGEN VAN EEN NIEUW BOEK

Omdat de API bij het toevoegen van een nieuw boek id nummers verwacht, was het een uitdaging om er voor te zorgen dat een gebruiker gewoon bij het toevoegen van een boek tekst kan typen, maar dat de API de juiste id nummers krijgt, of kan aanmaken. Ik heb terug gekeken in EdHub, maar dit was niet aanbod gekomen. Als startende frontend ontwikkelaar wordt het dan toch wel lastig om deze logica zelf te moeten bedenken. Hiervoor heb ik AI ingeschakeld om mijn leraar te zijn en me op weg te helpen. Het heeft resulteert een een hele lange `handleSubmit` functie in de `AddBooks.jsx` file. Hierin wordt er gekeken of de auteur en het genre al bestaan, en zo niet, dat deze worden aangemaakt. Ook hier heb ik weer gewerkt met verschillende API calls in één functie en zodoende geleerd dat je zowel een `axios.get` als `axios.post` in dezelfde functie kan verwerken. Zelf merk ik wel dat ik deze functie onoverzichtelijk vind. Wellicht doordat het een

behoorlijk lange functie is geworden. T.z.t. zal ik deze functie nogmaals willen bekijken, om na te gaan of deze toch ingekort kan worden.

Figuur 22

```
function AddBook() {  
  const handleSubmit = async () => {  
    try {  
      const searchResponseAuthor = await axios.get('https://nawi-backend-api-wpapi.onfigite.nl/ocwen/api/authors', {  
        params: {  
          name: authorOffice,  
        },  
        headers: {  
          'nawi-education-project-id': '168a4f5c-ae58-415a-a554-ae1eac9514d4',  
          'Content-Type': 'application/json',  
          'Accept': 'application/json'  
        }  
      });  
      let finalAuthorId;  
      if (searchResponseAuthor.data && searchResponseAuthor.data.length > 0) {  
        finalAuthorId = searchResponseAuthor.data[0].id;  
      } else {  
        const newAuthor = await axios.post('https://nawi-backend-api-wpapi.onfigite.nl/ocwen/api/authors', {  
          name: authorOffice,  
        }, {  
          headers: {  
            'nawi-education-project-id': '168a4f5c-ae58-415a-a554-ae1eac9514d4',  
            'Content-Type': 'application/json',  
            'Accept': 'application/json'  
          }  
        });  
        finalAuthorId = newAuthor.data.id;  
      }  
    }  
  }  
}
```

Noot. Screenshot code function AddBook/handleSubmit

INLOG/REGISTREER SCHERM

In mijn Figma ontwerp heb ik voor het inloggen en registreren een pop-up menu gebruikt. Echter lukte mij het nog niet om dit op de juiste manier in de praktijk uit te voeren, ondanks pogingen. Daarom heb ik er op dit moment voor gekozen om het inlog en registreer scherm in de gehele pagina te openen. Dit wil ik t.z.t. aanpassen naar een pop-up scherm, als mijn kennis hiervoor groter is geworden.

HET COMPONENT CURRENTBOOK

In mijn Figma ontwerp heb ik de lijst met boeken die op dit moment worden gelezen, qua styling anders dan in de werkelijke applicatie. Zo staan de afbeeldingen van de covers in het kader en verspringen ze niet iets, en heb ik ook (nog) niet de mogelijkheid ingebouwd om de voortgang van het boek dat op dit moment gelezen wordt bij te houden. Dit laatste heb ik geprobeerd in te bouwen, maar kostte me op dit moment teveel tijd om goed werkend te krijgen.

HET SORTEREN VAN DE LIBRARY

In eerste instantie was mijn idee om de Library alfabetisch te sorteren, met rechts een verticale balk met de letters van het alfabet, en dat je, zodra je op een letter klikt, naar het eerste boek gaat die met

die letter begint. Deze manier was nog iets te ambitieus, ik heb er nu voor gekozen om op drie verschillende manieren de Library te sorteren: Titel A-Z, Auteur A-Z en Genre A-Z. Hiervoor heb ik uiteindelijk gekozen voor een `switch` statement. Ik was begonnen met een `if/else` statement, maar merkte daarbij dat het voorbeeld wat ik had voor het sorteren van de “Tech-It-Easy” opdracht niet goed voldeed aan het sorteren van een lijst met boeken. Daarin ging het vooral om getallen teruggeven (a, b), terwijl ik nu een alfabetische lijst wilde hebben. Doordat auteurs en genres teruggeven worden door de API met een id nummer, moest ik hiervoor ook code schrijven om deze als tekst te kunnen laten sorteren. Een `switch` statement maakte hierdoor de functie overzichtelijker. Doordat ik strings met elkaar vergelijk, moest ik opzoek naar een methode om deze met elkaar te vergelijken. Google wees me op de functie `localeCompare` (*String - Javascript | MDN, 2025*) omdat deze alfabetisch sorteert en dit precies was wat ik wilde met mijn functie.

Figuur 23

```
const handleSort = (sortType) :void => { //tossavormedien07
  setActiveSort(sortType);
  if (!library) return;

  const sortedLibrary :any[] = [...library].sort((a, b) :any => {
    switch (sortType) {
      case 'author':
        const authorA = libraryAuthor?.find(auth => auth.id === a.authorId)?.name || "";
        const authorB = libraryAuthor?.find(auth => auth.id === b.authorId)?.name || "";
        return authorA.localeCompare(authorB);
      case 'genre':
        const genreA = genres?.find(g => g.id === a.genreId)?.name || "";
        const genreB = genres?.find(g => g.id === b.genreId)?.name || "";
        return genreA.localeCompare(genreB);
      case 'title':
      default:
        return a.title.localeCompare(b.title);
    }
  });
  setLibrary(sortedLibrary);
}
```

Noot. Screenshot functie `handleSort`

Limitaties

COVER VAN HET BOEK MEESTUREN

Het liefst had ik het ook voor elkaar gekregen dat bij het toevoegen van een nieuw boek, ook de cover meegestuurd zou worden. Echter is daar meer backend kennis voor nodig, wat ik op dit moment nog onvoldoende beheers. Een ontwikkelpunt voor mij, maar gezien het feit dat ik de bootcamp AI-Developer doe, en niet de fullstack bootcamp, heb ik deze functionaliteit voor nu geparkeerd, maar wordt deze zeker niet van de lijst gehaald.

HET TOEVOEGEN VAN BOEKEN AAN CURRENTLYREADINGLIST

Bij de gebruiker die in de `data.json` file is toegevoegd, staan boeken die op dit moment gelezen worden. Echter kan een nieuwe gebruiker op dit moment nog geen boeken toevoegen aan deze lijst.

Omdat ik bij mijn ideefase in eerste instantie alleen had aangegeven dat de gebruiker kon aangeven welke boeken waren gelezen en het me veel tijd kostte om deze functie goed werkend te krijgen, heb ik vooralsnog de functie van het toevoegen aan de `currentlyReadingList` even gelaten voor wat het is.

HET GEVEN VAN HARTJES AAN BOEKEN

Deze functionaliteit heb ik deels toegevoegd, het is een start van een extraatje, gezien dit niet specifiek in mijn ideefase stond. Het geven van sterren/hartjes is voor mij onlosmakelijk verbonden met het schrijven van een review, echter heb ik deze functie nog niet goed werkend gekregen, dat deze ook bij de verschillende componenten te gelijk te zien zijn (onder het boek, bij het review, in de bibliotheeklijst etc.). Onder het boek wordt het wel opgeslagen in de State, maar ik krijg het nog niet werkend naar de backend, dat het ook bij de reviews staat, wat elke user heeft gegeven. Ik zie bijvoorbeeld bij mijn account dat ik 5 hartjes aan het boek heb gegeven, maar in de `data.json` staat er bij de rating “4”. Om de heart-rating te krijgen zoals ik hem nu heb, heb ik gegoogled en ik kwam op YouTube op deze video terecht: <https://youtu.be/eDw46GYAIDQ?si=kdANfaa6qtayGGFP> (Murphy, E. 2020). Deze tutorial heb ik gebruikt om de heart-rating te bouwen.

HET AANMAKEN VAN LIJSTEN DOOR DE GEBRUIKER

De enige twee lijsten die er op dit moment zijn, zijn de lijsten met “gelezen” en “op dit moment aan het lezen”. Ik wil de gebruiker de mogelijkheid geven om zelf ook nog lijsten aan te maken waar ze boeken in kunnen plaatsen, zoals bijvoorbeeld een “did-not-finish” lijst. Mijn kennis is op dit moment nog onvoldoende om zoiets te kunnen toevoegen, omdat dit ook backend kennis vereist. Daarnaast moest ik echt keuzes maken welke onderdelen ik wel/niet ging implementeren. Juist de vier uit mijn ideefase, kostte mij al behoorlijk wat tijd om goed werkend te krijgen.

HET TOEVOEGEN VAN QUOTES UIT BOEKEN

In de `data.json`, de uitgezette code in de `Home.jsx` en de `NOVI API` is al te zien dat ik een start gemaakt heb met het toevoegen van quotes. Omdat de vier verplichte functionaliteiten mij al veel tijd hebben gekost, heb ik dit extraatje niet verder uitgewerkt, maar dit staat zeker nog op de planning om door te ontwikkelen.

PROFILE PAGE/ACCOUNTSETTINGS WIJZIGEN

Tijdens de lessen hebben we wel geleerd hoe we een gebruiker kunnen laten registreren en inloggen, maar bij de desbetreffende huiswerkopdracht klopte er iets niet bij het toevoegen van gegevens van de ingelogde gebruiker op een pagina door dat er iets niet in de API stond. Op dit moment wordt alleen het email adres dynamisch opgehaald. De naam en het lid sinds wat ik wilde toevoegen, lukt me alleen nog maar hard-coded.

READING STATS EN READING GOALS

In mijn scherm ontwerp zijn op de homepage Reading Stats en Reading Goals te zien. Deze heb ik op dit moment in mijn applicatie nog niet kunnen toevoegen. Door dit op een gegeven moment toe te gaan voegen, hoop ik de applicatie interactiever te kunnen maken voor de gebruikers.

GitHub Repository Link

Hieronder de link naar de openbare GitHub Repository Link:

<https://github.com/tessavermeulen07/boekify>

Hierbij meld ik gelijk dat ik pas te laat zag dat ik al meermalen mijn pull requests had moeten mergen. Hier heb ik totaal niet meer aandacht tijdens het maken van mijn applicatie. Ik heb dus maar 1 pull rekest gedaan en toen alles klaar was het gemerged. Ik merk dan toch dat ik wel goed weet hoe ik een branch kan maken, het naar de branch kan pushen, maar het mergen niet in mijn systeem zit, omdat dit bij de huiswerkopdrachten niet echt nodig was.

Bijlagen

- AI prompts

AI Prompts

Hiervoor heb ik gebruik gemaakt van Gemini van Google.

Link: <https://gemini.google.com/share/5e977e0eb3a6>

Prompt:

Jij bent mijn docent, je zegt niks voor, je geeft tips.

Dit is mijn code:

```
import './Library.css';
import Navigation from '../navigation/Navigation.jsx';
import axios from 'axios';
import {useEffect, useState} from 'react';
```

```
function Library() {

  const [library, setLibrary] = useState(null);
  const [loading, toggleLoading] = useState(false);
  const [error, toggleError] = useState(false);
  const [libraryAuthor, setLibraryAuthor] = useState(null);

  async function getBooks() {
    try {
      const resultBooks = await axios.get('https://novi-backend-api-wgsgz.ondigitalocean.app/api/
books', {
        headers: {
          'novi-education-project-id': '268aff3c-ae58-411a-a55f-e0c1ec05146d'
        }
      });

      resultBooks.data.sort((a, b) => {
        const titleA = a.title;
        const titleB = b.title;

        if (titleA < titleB) {
          return -1;
        } else if (titleA > titleB) {
          return 1;
        } else {
          return 0;
        }
        // return a.title - b.title
      })

      setLibrary(resultBooks.data);
```

```

        console.log(resultBooks);
    } catch (e) {
        console.error(e);
    }
}

useEffect(() => {
    void getBooks();
}, []);

async function getAuthors() {
    try {
        const resultAuthors = await axios.get('https://novi-backend-api-wgsgz.ondigitalocean.app/api/authors', {
            headers: {
                'novi-education-project-id': '268aff3c-ae58-411a-a55f-e0c1ec05146d'
            }
        })

        setLibraryAuthor(resultAuthors.data);
        console.log(resultAuthors);
    } catch (e) {
        console.error(e);
    }
}

useEffect(() => {
    void getAuthors();
}, []);

return (
    <
    <Navigation/>

    <div className="main-content-library">
        <div className="left-content-library">
            <div className="inside-left-content-library">
                <h3>Persoonlijke Filters</h3>
                <p>Link</p>
                <p>Link</p>
            </div>
            <div className="inside-left-content-library">
                <h3>Boek Filters</h3>
                <p>Link</p>
                <p>Link</p>
            </div>
        </div>
    </div>
    </

```

```

    </div>
  </div>
  <div className="right-content-library">
    <div className="inside-right-content-library">
      <ul>
        {library?.map((books) => {
          return (
            <li key={books.title} className="list-books-library">
              <p className="title-books-library">{books.title}</p>
              <p className="author-books-library">{books.authorId}</p>
            </li>
          );
        })}
      </ul>
    </div>
  </div>
</div>
</>
)
}

```

export default Library;

Ik wil graag in plaats van de authorId waarbij het cijfer te zien is, graag de naam zien. Ik weet dat ik hiervoor de find methode kan gebruiken bij libraryAuthor, maar ik krijg het niet goed werkend op de pagina.

Antwoord:

Dag! Goed dat je zelf al de link legt naar de `.find()` methode. Je hebt de twee ingrediënten die je nodig hebt al in je state staan: de lijst met boeken (met een `authorId`) en de lijst met auteurs (met hun namen en ID's).

Om dit werkend te krijgen op je pagina, help ik je graag op weg met een paar kritische vragen en tips:

1. Waar vindt de koppeling plaats?

Je probeert nu in je JSX (tussen de `<ul>` tags) de naam te tonen. Op die plek heb je per boek alleen de `books.authorId`.

- **Tip:** Binnen de `.map()` van je boeken kun je voor elk individueel boek op zoek gaan in de `libraryAuthor` array. Als je daar een auteur vindt wiens ID overeenkomt met de `authorId` van het boek, heb je het juiste object te pakken.

2. De timing van je data

Je haalt beide lijsten op met `useEffect`. Omdat dit asynchroon gaat, kan het zijn dat de boeken al geladen zijn, maar de auteurs nog niet (of andersom).

- **Tip:** Controleer in je `JSX` altijd of `libraryAuthor` wel al gevuld is voordat je er een `.find()` op loslaat. Wat gebeurt er in je code als `libraryAuthor` nog `null` is terwijl de boeken al op het scherm verschijnen?

3. De vergelijking

Bij het gebruik van `.find()` vergelijk je twee waarden met elkaar.

- **Tip:** Let goed op de datatypes. Is de `authorId` uit je boeken-lijst een `String` of een `Number`? En hoe zit dat in de auteurs-lijst? Gebruik `console.log` om te checken of je wel appels met appels vergelijkt.

Een kleine uitdaging voor je logica:

Stel dat je de `.find()` binnen de `map` uitvoert, dan ziet dat er ongeveer zo uit:

1. Pak een boek.
2. Zoek in `libraryAuthor` naar de auteur met `id === books.authorId`.
3. Sla dit resultaat op in een variabele (bijv. `const author = ...`).
4. Toon `author.name` (of hoe de naam-property ook heet in jouw API) in de `<p>` tag.

Vraag aan jou: Hoe zou je die `const author` definiëren binnen de `map` functie, rekening houdend met het feit dat de data nog aan het laden kan zijn?

Zal ik eens met je meekijken naar hoe je die `.find()` nu precies had opgeschreven?

Prompt:

Jij bent mijn docent, je zegt niks voor, maar helpt me op weg.

Ik heb een bookrating systeem doormiddel van hartjes, deze werkt, maar bij een refresh van de pagina verdwijnt deze weer. Het wordt niet opgeslagen in de state. Wat mis ik?

Antwoord:

Goed dat het visueel al werkt met die hartjes! Dat is vaak de eerste grote hobbel die je moet nemen.

Ik wil je eigen aanname echter even uitdagen. Je zegt: *"Het wordt niet opgeslagen in de state."*

Maar bedenk eens: als de hartjes correct oplichten wanneer je erop klikt en dit goed op je scherm blijft staan *totdat* je de pagina ververs, dan zit het hoogstwaarschijnlijk wél goed in je state opgeslagen!

Het probleem ligt ergens anders. Wat denk je dat een webbrowser doet met het werkgeheugen (en dus met jouw React state) op het moment dat jij op 'vernieuwen' drukt?

React state is eigenlijk het kortetermijngeheugen van je applicatie. Zodra de pagina ververs, wordt de hele applicatie opnieuw ingeladen en beginnen al je `useState` hooks weer bij hun beginwaarde (vaak `null`, `false`, of een lege array).

Om gegevens een refresh te laten overleven, moet je het opslaan in een 'langetermijngeheugen'. Hier zijn grofweg twee smaken voor:

1. **De Database (via een API):** Je haalt de boeken nu al op uit de NOVI backend met een GET request. Biedt deze API toevallig ook een route (bijvoorbeeld via een POST, PUT of PATCH request) om een rating van een gebruiker op te sturen en veilig in de database te zetten?
2. **De Browser Opslag:** Als de API geen mogelijkheid biedt om ratings op te slaan, heeft de webbrowser zelf ook een soort 'kluisje' ingebouwd. Hiermee kun je data op de computer van de bezoeker bewaren, zodat het er na een refresh gewoon nog staat. Weet je toevallig hoe dit kluisje in de browser heet?

Hoe wil je dit het liefst aanpakken, of wat wordt er verwacht vanuit je opdracht: wil je het opslaan in de externe database, of lokaal in de browser van de gebruiker?

Prompt:

De mogelijkheid voor de API is er wel, maar ik denk dat op dit moment de localStorage makkelijker is. Dus dan moet ik localStorage gebruiken.

Antwoord:

Je hebt inderdaad gelijk dat localStorage een manier is om gegevens in de browser te bewaren, zodat ze een refresh overleven. Toch ga ik hier niet zomaar met je instemmen om voor de makkelijkste weg te kiezen.

Ik wil je keuze juist even uitdagen, want er zitten een paar flinke nadelen aan localStorage voor een functionaliteit zoals het beoordelen van boeken:

1. **Apparaat-afhankelijk:** Stel, je bouwt deze applicatie en je opent hem straks vol trots op je eigen telefoon om hem aan iemand te laten zien. Wat denk je dat er met jouw zorgvuldig uitgedeelde hartjes is gebeurd? Die staan namelijk in de localStorage van de browser op je laptop, en *niet* op je telefoon.
2. **Geen gedeelde data:** Als je later in je project (of in een echte applicatie) een gemiddelde rating van een boek wilt tonen op basis van wat *alle* gebruikers vinden, dan kan dat niet. De backend weet immers van niks.

Je geeft zelf al aan dat de mogelijkheid in de API er wel is. Bovendien zie ik in je eerdere code dat je al ontzettend goed snapt hoe je met axios een beveiligde verbinding (met headers) maakt naar de NOVI-backend. Je hebt het moeilijkste deel van het werken met API's dus al onder de knie!

Mijn advies als docent? Grijp die API met beide handen aan. Het is een veel realistischere en robuustere oplossing voor je portfolio.

Als je de uitdaging met de API aan wilt gaan (en ik hoop echt van wel!), is mijn eerste vraag aan jou:

Welk type HTTP-request (naast de GET die je al kent) denk je dat je nodig gaat hebben om informatie op te sturen of te wijzigen in een database?

Mocht je toch écht bij je standpunt blijven en `localStorage` willen gebruiken, geef het gerust aan, dan help ik je via die route verder op weg. Maar denk er even goed over na!

Prompt:

Moet ik dan in de API een aparte rating sectie maken?

Antwoord:

Goede vraag! Het hangt er een beetje vanaf hoe de "architect" van de API het heeft bedacht. Als frontend-developer "maak" je meestal geen secties *in* de API (dat doet de backend-developer), maar je kijkt naar wat er beschikbaar is.

Laten we eens logisch naar de twee meest voorkomende scenario's kijken:

Scenario A: De Rating is onderdeel van het Boek

In dit geval heeft elk boek-object in de database al een veldje, bijvoorbeeld `rating` of `stars`.

- **Hoe werkt het?** Je stuurt een seintje naar de API om een *bestaand* boek aan te passen.
- **Vraag voor jou:** Welk HTTP-request gebruik je als je een bestaand onderdeel in een database wilt **updaten** of wijzigen? (Hint: het is niet `GET` en ook niet `POST`).

Scenario B: De Rating is een losstaand object

Soms wil een API bijhouden *wie* welke rating geeft. Dan is er vaak een apart eindpunt, bijvoorbeeld `/api/ratings`.

- **Hoe werkt het?** Je stuurt een **nieuw** rating-object op dat gekoppeld is aan het `id` van het boek.
- **Vraag voor jou:** Welk HTTP-request gebruik je als je iets **nieuws** wilt toevoegen aan een database?

Jouw actieplan:

Om te weten wat je moet doen, moet je de "menukaart" van je API bekijken (vaak de **Swagger** documentatie of een README-bestand van je opdracht).

1. Kun je in de documentatie van `https://novi-backend-api-wgsgz.ondigitalocean.app` een endpoint vinden waar je data naartoe kunt sturen (een `POST`, `PUT` of `PATCH`)?
2. Kijk specifiek of er bij de boeken-sectie iets staat over het aanpassen van een boek, of dat er een aparte sectie is voor reviews/ratings.

Mijn tip voor nu: Ga naar de documentatie-pagina van deze API (meestal gewoon de URL in je browser plakken zonder `/api/books` erachter). Welke andere acties zie je daar staan naast de `GET` die je al gebruikt?

Zodra je weet welke "methode" de API verwacht, kunnen we kijken hoe je die hartjes-klik omzet in een seintje naar de server! Wat zie je in de documentatie staan?

Prompt:

Jij bent mijn leraar. Ik wil graag hulp (zonder voorzeggen) hoe ik het plaatsen van een review bij een boek dynamisch kan doen, dus zodat `authorId`, `memberId`, `bookId` etc al ingevuld zijn. En hoe ik de rating via de hartjes kan opslaan, want dat gebeurt op het moment niet.

Antwoord:

Goed dat je de stap maakt naar een dynamisch systeem! Dat is precies waar de kracht van React ligt: componenten die "weten" voor welk specifiek item ze aan het werk zijn.

Laten we dit opsplitsen in twee delen: het verzamelen van de juiste ID's en het daadwerkelijk opslaan van de rating.

1. De "Context" van je component (De ID's)

Om een review te plaatsen, heeft de API specifieke informatie nodig. Je noemde `authorId`, `memberId` en `bookId`.

- **De Book ID:** In je `Library` component loop je met `.map()` door je boeken. Elk `books` object bevat waarschijnlijk al een id.
 - **Tip:** Hoe zorg je ervoor dat je `BookRating` of `Review` component weet bij welk boek hij hoort? Denk aan het doorgeven van **props**. Als je het boek-ID als prop meegeeft, kan de child-component dit direct gebruiken in het Axios-request.
- **De Member ID:** Dit is meestal de ID van de persoon die momenteel is ingelogd.
 - **Tip:** Heb je al een plek waar je de gegevens van de ingelogde gebruiker bewaart (bijvoorbeeld een `AuthContext`)? Zo ja, dan kun je die daaruit "tappen". Zo niet, staat deze ID misschien ergens in je `localStorage` na het inloggen?

2. De Rating opslaan via de hartjes

Je zegt dat het opslaan nu niet gebeurt. Laten we kijken naar de actie-reactie keten:

1. **De Trigger:** De gebruiker klikt op een hartje.
2. **De State:** Je lokale state (bijv. `rating`) wordt geüpdatet zodat de hartjes visueel veranderen.
3. **De API-call:** *Hier gaat het vaak mis.* Gebeurt je `axios.post` direct in de functie die wordt aangeroepen door de `onClick` van het hartje?

Checklist voor je POST-request:

- **De Body:** De API verwacht een specifiek object. Ziet jouw "payload" er precies zo uit als wat de documentatie voorschrijft? (Bijv: `{ "bookId": 1, "rating": 5, ... }`).
- **De Headers:** Vergeet niet dat je voor schrijfacties (POST) vaak dezelfde headers (zoals je project-id of een token) nodig hebt als bij je GET-request.

3. Waarom zie je het niet na een refresh?

Als je `POST` succesvol is (check je **Network tab** in de browser!), dan staat de data in de database. Maar als je de pagina ververs, haalt je `Library` component de boeken opnieuw op met een `GET`-request.

- **Vraag voor jou:** Als je in de Network tab naar het resultaat van je `GET` request kijkt na de refresh, zie je daar dan de nieuwe rating bij het boek staan?
 - **Nee?** Dan is je `POST` waarschijnlijk niet gelukt of niet volledig.
 - **Ja?** Dan gebruik je in je code waarschijnlijk nog een "hardcoded" beginwaarde voor je hartjes in plaats van de waarde die uit de API komt.

Mijn tip voor de volgende stap:

Kijk eens in je `BookRating` component naar de functie die wordt afgevuurd als je op een hartje klikt. Staat daar al een `axios.post` in, of pas je daar alleen de lokale state aan?

Zal ik je helpen kijken naar hoe je die `POST` body kunt opbouwen met de gegevens die je uit de props haalt?

Prompt:

Jij bent mijn leraar, je geeft tips maar zegt niks voor. Als ik inlog op mijn pagina, krijg ik deze melding: `installHook.js:1 TypeError: login is not a function at handleSubmit (Login.jsx:34:13)`. Na refresh van de pagina kan ik wel inloggen.

Antwoord:

Vervelend, die foutmeldingen! Maar ze vertellen ons vaak precies waar we moeten zoeken. Een `TypeError: login is not a function` betekent simpelweg dat je probeert iets uit te voeren (met haakjes erachter: `login()`), terwijl JavaScript op dat moment zegt: "Ho even, dit is helemaal geen functie, dit is `undefined` (of iets anders)."

Het feit dat het **na een refresh** wel werkt, is een hele belangrijke aanwijzing. Dat betekent dat de logica wel klopt, maar de *timing* of de *initialisatie* niet.

Laten we samen op onderzoek uitgaan zonder het voor te zeggen. Kijk eens naar deze drie punten:

1. Waar komt `login` vandaan?

In je `Login.jsx` haal je `login` waarschijnlijk ergens vandaan. Is dit uit een **Context** (zoals een `AuthContext`)?

- **Tip:** Controleer hoe je de functie binnenhaalt. Gebruik je `useContext`? Als je `const login = useContext(AuthContext)` schrijft, maar je Context-object is een object met meerdere dingen erin (zoals `{ login, logout, user }`), dan is `login` op dat moment het hele object in plaats van alleen de functie.
- **Vraag voor jou:** Gebruik je de juiste *destructuring* (de krulhaken `{ }`) bij het ophalen van de functie?

2. De beginwaarde van je Provider

Kijk eens naar de plek waar je de `AuthProvider` hebt gedefinieerd.

- **Tip:** Wat is de `defaultValue` die je aan `createContext()` hebt meegegeven? Als die leeg is of op `null` staat, en je component probeert `login` aan te roepen voordat de Provider de echte functie heeft klaargezet, dan krijg je deze fout.
- **Vraag voor jou:** Kan het zijn dat je in je `handleSubmit` de functie probeert aan te roepen op een moment dat de Context nog niet volledig is geïnitieerd?

3. Waarom werkt het na een refresh wel?

Dit is de "smoking gun". Na een refresh wordt je hele app opnieuw opgebouwd.

- **Tip:** Vaak gebruiken we `useEffect` of `localStorage` om een gebruiker ingelogd te houden. Als die logica bij de eerste keer laden (zonder refresh) nog niet is afgerond, kan de `login`-functie misschien nog niet beschikbaar zijn gesteld aan de rest van de app.

Je volgende stap:

Om de fout te vinden, moet je weten wat `login` precies is op het moment van de crash. Zet eens een `console.log(login)` vlak boven je `handleSubmit` (of helemaal bovenaan in je component).

Wat zie je in de console verschijnen op het moment dat je de pagina voor het eerst opent? Is het een `function`, of staat er `undefined`?

Als je wilt dat ik even met je meekijk naar de "bron" van de functie, laat me dan zien hoe je `login` importeert of ophaalt in `Login.jsx`!

Prompt:

Jij bent nog steeds mijn leraar. Ik probeer het ophalen van de data van de `CurrentReads` dynamisch aan het maken, maar als ik ingelogd ben, krijg ik de melding dat de boeken niet gevonden worden en een 404 melding bij de GET request.

Antwoord:

Een 404-melding (Not Found) bij een GET-request is een heel specifiek signaal. Het betekent dat de browser braaf naar het adres is gegaan dat jij hebt opgegeven, maar dat de server zegt: "Op deze plek heb ik niks voor je."

Omdat je zegt dat dit gebeurt wanneer je de data **dynamisch** probeert op te halen, is de kans groot dat er iets misgaat in de opbouw van je URL. Laten we als detectives naar de volgende drie punten kijken:

1. De Inspecteur-methode (Network Tab)

Dit is je belangrijkste gereedschap. Open je Developer Tools (F12) en ga naar het tabblad **Network**. Ververs de pagina zodat de fout optreedt en klik op de rode GET-aanvraag die faalt.

- **De URL-check:** Kijk bij 'Request URL'. Hoe ziet die er *exact* uit?
- **Tip:** Zie je daar ergens het woord `undefined`, `null` of een lege plek staan op een plek waar je een ID of naam had verwacht? Bijvoorbeeld: `.../api/users/undefined/reads`. Als dat zo is, weet je dat je variabele op het moment van verzenden nog niet gevuld is.

2. De bron van je ID

Je probeert de `CurrentReads` waarschijnlijk op te halen op basis van een `memberId` of `userId`.

- **Vraag voor jou:** Waar haal je dat ID vandaan? Komt dit uit je `AuthContext` of uit de `localStorage`?
- **De Timing-valstrik:** In React gaat het ophalen van Context-data soms net iets trager dan het uitvoeren van een `useEffect`. Als je `useEffect` al begint te rennen terwijl je `user`-object nog `null` is, probeert hij een request te doen naar een ongeldig adres.
- **Tip:** Hoe kun je je `useEffect` vertellen dat hij pas echt aan de slag mag gaan als het ID ook daadwerkelijk beschikbaar is? (Denk aan de `dependency array` en een simpele `if-check`).

3. De API-structuur

Kijk nog eens goed in de documentatie van de NOVI-backend (Swagger).

- **Vraag voor jou:** Gebruik je voor de dynamische route wel exact de juiste pad-opbouw? Soms vergeten we een schuine streep (/) of gebruiken we per ongeluk `books` waar de API `current-reads` verwacht.
- **Check:** Komt de structuur van de URL in je code 100% overeen met wat de documentatie voorschrijft voor het ophalen van persoonlijke leeslijsten?

Jouw volgende actie:

Kijk in je code naar de regel waar je de URL voor de `axios.get` samenstelt (waarschijnlijk iets met *backticks* zoals ``.../${id}``).

Zet eens een `console.log` vlak boven je `axios`-aanroep:

```
console.log("Ik ga nu data ophalen voor ID:", id);
```

Wat zie je in je console verschijnen vlak voordat de 404-fout in beeld komt? Is de waarde van `id` wat je ervan verwacht?

Prompt:

Jij bent nog steeds mijn leraar. Hoe kan ik op een handige manier ervoor zorgen dat een gebruiker in een invoerveld om een boek toe te tekst kan gebruiken voor het genre en de auteur waar de API een id nummer verwacht? En als de auteur of het genre nog niet in de database staat, dat deze wordt toegevoegd?

Antwoord:

Dit is een klassiek probleem in webdevelopment: hoe maak je een interface die gebruiksvriendelijk is voor mensen (tekst), maar ook technisch correct voor de computer (ID's)?

Omdat je leraar ben, ga ik je niet de code geven, maar wel de **architectuur** die je nodig hebt om dit probleem op te lossen. Je zult hier een "kettingreactie" van logica moeten bouwen.

1. De "Vertaalmachine" (De Opzoekfase)

Je hebt in je state waarschijnlijk al lijsten met auteurs (`libraryAuthor`) en genres. Wanneer een gebruiker een naam typt (bijvoorbeeld "J.K. Rowling"), moet jouw code eerst gaan zoeken.

- **Tip:** Gebruik de `.find()` methode op je bestaande lijst. Zoek naar een object waar de naam precies overeenkomt met de input van de gebruiker.
- **Resultaat:** Heb je een match? Dan heb je het **ID** al te pakken en ben je klaar voor de volgende stap. Heb je géén match? Dan weet je dat dit een nieuwe auteur/genre is.

2. De "Ketting-logica" (Async/Await volgorde)

Wanneer de gebruiker op "Boek Toevoegen" klikt, kun je niet meteen het boek opslaan. Je moet een volgorde aanhouden. Dit noemen we vaak een **Conditional POST**.

De stappen in je functie:

1. **Check auteur:** Bestaat de auteur al?
 - *Ja:* Gebruik het bestaande ID.
 - *Nee:* Doe eerst een `axios.post('/authors', ...)` en wacht op het antwoord. De API stuurt je als het goed is het **nieuwe ID** van deze auteur terug. Sla dit ID tijdelijk op.
2. **Check genre:** Doe precies hetzelfde voor het genre.
3. **Het boek opslaan:** Pas als je beide ID's hebt (of ze nu oud of nieuw zijn), doe je de uiteindelijke `axios.post('/books', ...)` met die ID's in de body.

3. Gebruikersgemak: De Datalist

Wil je voorkomen dat mensen tikfouten maken (waardoor je per ongeluk "J.K. Rowlin" én "J.K. Rowling" in je database krijgt)?

- **Tip:** Kijk eens naar het HTML `<datalist>` element. Dit werkt als een combinatie tussen een invoerveld en een dropdown. De gebruiker kan typen, maar krijgt ook suggesties uit je bestaande lijst te zien. Als ze een suggestie aanklikken, weet je zeker dat de tekst overeenkomt met een bestaand ID.

Een kritische vraag voor je logica:

Stel dat je in je `handleSubmit` functie drie `axios.post` aanroepen onder elkaar zet. Hoe zorg je ervoor dat de tweede en derde aanroep pas gebeuren als de eerste (het aanmaken van de auteur) helemaal klaar is?

Hint: Denk aan het gebruik van `await` en hoe je de resultaten van de eerste aanroep opvangt in een variabele.

Wat denk je, lukt het je om deze "stappenvisie" om te zetten in een functie met een paar `if/else` blokken? Of zal ik je helpen met hoe je die volgorde in `async/await` precies structureert?

Prompt:

Jij bent mijn leraar, je zegt niets voor, je geeft tips.

Ik wil door middel van een `select` de status van het boek veranderen, zowel in de State (dus dat het op de pagina te zien is: gelezen, ongelezen, op dit moment aan het lezen), als dat het boek gezet wordt in de `readList`, of in de `currentlyReadingList`, in de API. Om het boek van lijsten te laten veranderen, zal ik het boek eerst uit de desbetreffende lijst moeten verwijderen, om het daarna toe te voegen, en als het nog niet in een lijst staat, moet ik het boek daaraan toevoegen. Maar ik ben even blanco hoe ik dat met de code handig kan doen. Moet ik een `<form>` tag om de `<select>` tag plaatsen? Of kan een `handleStatusChange` doen?

Antwoord:

Dat is een mooie puzzel waar je voor staat. Je raakt hier aan de kern van **State Management** in combinatie met **API-orchestratie** (het in de juiste volgorde aanroepen van verschillende eindpunten).

Laten we je vragen één voor één langslopen om de mist in je hoofd te laten verdwijnen.

1. Form of Event Handler?

Je vraagt je af of er een `<form>` omheen moet. In "modern" React is een `<form>` vooral handig als je een hele verzameling gegevens tegelijk wilt versturen (zoals een compleet profiel of een nieuw boek).

Voor een snelle statuswijziging van een enkel item is een **onChange handler** op je `<select>` veel directer en soepeler. Zodra de gebruiker een optie kiest, wordt je functie `handleStatusChange` afgevuurd en kun je direct actie ondernemen. Geen "Submit"-knop nodig!

2. De logica van het 'Verplaatsen'

Je gaf zelf al aan dat je een boek eerst uit de ene lijst moet verwijderen om het in de andere te zetten. Dit noemen we vaak een **transactie**.

- **Stap A (De Check):** Je moet weten waar het boek *nu* staat. Heb je die informatie al in je `books` object? Bijvoorbeeld een property `currentStatus`?
- **Stap B (De Verwijdering):** Als het boek in een lijst staat, moet je een `DELETE` request doen naar dat specifieke lijst-item.
- **Stap C (De Toevoeging):** Daarna doe je een `POST` request naar de nieuwe lijst.

3. Hoe doe je dit 'handig'?

Als je dit in je code gaat schrijven, loop je tegen het probleem aan dat je meerdere asynchrone taken achter elkaar moet doen.

Mijn tips voor de aanpak:

- **Gebruik een switch of if/else blok:** Binnen je `handleStatusChange` kun je kijken naar de `e.target.value`. Op basis van die waarde bepaal je welke API-endpoints je moet aanspreken.
- **Sequential Await:** Gebruik `await` voor je verwijdering voordat je de toevoeging doet. Zo voorkom je dat de database in de war raakt.
- **Optimistic UI vs. Re-fetching:** * *Optie 1:* Je doet de API-calls en als ze klaar zijn, roep je je `getBooks()` functie opnieuw aan om de hele pagina te verversen. (Makkelijkst).
 - *Optie 2:* Je past de lokale state direct aan (`setLibrary(...)`) zodat de gebruiker meteen resultaat ziet, terwijl op de achtergrond de API wordt bijgewerkt. (Sneller, maar iets complexer).

Een vraag om je op weg te helpen:

Hoe weet jouw component op dit moment in welke lijst een boek staat? Krijg je die informatie al mee vanuit de API (bijvoorbeeld: `isInReadList: true`), of moet je dat zelf nog uitzoeken door de verschillende lijsten met elkaar te vergelijken?

Welke methode denk je dat het meest overzichtelijk blijft in je `handleStatusChange` functie?

Prompt:

Jij bent mijn leraar, je helpt me op weg. Ik heb een `currentlyReadingList` en ik heb een `ReadList` in de API. Ik heb code gemaakt om de leesstatus van een boek op de boekpagina te laten zien. Ik heb een handler geschreven om de leesstatus te veranderen van op dit moment aan het lezen naar gelezen, of van ongelezen naar op dit moment aan het lezen etc. Ik krijg geen foutmeldingen meer wanneer de API wordt opgeroepen het boek te verwijderen, of te plaatsen in de goede lijst, maar de `checkBookStatus` (die in de handler wordt aangeroepen) verandert niet en bij een refresh staat het boek nog steeds in dezelfde lijst.

Antwoord:

Dat is een klassiek "detective-probleem" in development! Als je geen foutmeldingen krijgt (de API zegt 200 OK of 204 No Content), maar er verandert niets in de database én niet in je state, dan is er sprake van een **stille mismatch**.

Laten we de drie meest waarschijnlijke scenario's onderzoeken:

1. De "Succesvolle" Niks-doener

Soms geeft een API een succesmelding terug, ook al is er feitelijk niets gebeurd.

- **De Theorie:** Stel dat je een boek probeert te verwijderen uit de `ReadList`, maar je stuurt per ongeluk een verkeerd `id` of een verkeerde `memberId` mee. De API vindt het item niet, maar vindt dat geen "fout" en zegt gewoon: "Klaar!".
- **De Check:** Kijk in je **Network Tab** bij de `POST` of `DELETE` aanroep. Open het tabblad **Payload**. Zijn de ID's die je daar ziet staan exact gelijk aan de ID's die je ziet bij je `GET` request van die lijst?

2. De Async Timing-valstrik

Je geeft aan dat `checkBookStatus` in de handler wordt aangeroepen. In React zijn API-calls asynchroon.

- **De Theorie:** Als je code er zo uit ziet:
JavaScript

`deleteBook();`
- `addBookToNewList();`
`checkBookStatus();`
- Dan begint JavaScript aan de eerste twee, maar wacht niet tot ze klaar zijn voordat `checkBookStatus` alweer wordt uitgevoerd. De state wordt dan gecontroleerd *voordat* de database is bijgewerkt.
- **De Check:** Heb je overal `await` voor staan? En is je `handleStatusChange` zelf gemarkeerd als `async`?

3. De Identiteitscrisis van het Boek

Bij een refresh haal je de data opnieuw op. Als het boek dan nog steeds in de oude lijst staat, is de `POST` of `DELETE` simpelweg niet doorgevoerd in de database.

- **De Theorie:** Gebruik je de juiste headers? In je eerdere code zag ik een `novi-education-project-id`.
- **De Check:** Als je een boek toevoegt aan de `ReadList`, vereist de API dan misschien een specifiek object-format? Als je bijvoorbeeld `{ id: 1 }` stuurt terwijl de API `{ bookId: 1 }` verwacht, negeert de server je verzoek soms zonder een foutmelding te geven.

Jouw volgende stap als detective:

We gaan even uitsluiten of het aan je React-code ligt of aan de communicatie met de server.

Doe de actie (verander de status), wacht tot de aanroepen klaar zijn en open dan een **nieuw tabblad** in je browser. Ga daar direct naar de URL van je `ReadList` (bijv. `.../api/members/{id}/read-list`).

Zie je daar het boek wel of niet tussen staan? Als het daar *niet* staat, weten we dat we niet naar je `checkBookStatus` hoeven te kijken, maar naar de inhoud van je `POST` of `DELETE` request. Wat zie je in dat nieuwe tabblad?

Bronvermelding

1. *Screenshots van Hardcover.* (z.d.). Hardcover. <https://hardcover.app/>
2. *Screenshots van Goodreads.* (z.d.). Goodreads. <https://www.goodreads.com/>
3. *Screenshot van Bookly.* (z.d.). Bookly. <https://getbookly.com/>
4. *Screenshots van Book Tracker.* (z.d.). Book Tracker. <https://booktrack.app/>
5. Eric Murphy. (2020, 28 januari). *How to Make a Star Rating Component in React* [Video]. YouTube. <https://www.youtube.com/watch?v=eDw46GYAIDQ>
6. *String - Javascript | MDN.* (2025, 10 juli). https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/
7. *Wireframes.* (2025). Tessa Vermeulen
8. *Screenshots code.* (2026). Tessa Vermeulen
9. *AI Gemini.* (z.d.). <https://gemini.google.com>
10. *Scribbr.* (z.d.). www.scribbr.nl
11. *Bookcovers.* (z.d.). Goodreads. <https://www.goodreads.com/>
12. *Icons.* (z.d.). Flaticon. <https://www.flaticon.com/>
(aangepast met de juiste kleur door Tessa Vermeulen)
13. *Afbeeldingen op de startpagina en de registreerpagina.* (z.d.). AI imagetooi. Canva.
<https://www.canva.com>